

SHELL SCRIPTING

A shell is a command-line interpreter that provides an interface between the user and the operating system. It acts as a mediator that accepts user commands, interprets them, and forwards them to the system kernel for execution. The shell enables users to perform various operations such as file handling, program execution, and system management.

Shell scripting refers to the process of writing a sequence of commands in a file, known as a script, which can be executed automatically. Instead of typing commands manually each time, users can store them in a script file and execute them whenever required, thereby improving efficiency and reducing manual effort.

Types of Shells

There are several types of shells available in Unix/Linux systems. The most commonly used shells include:

- Bash (Bourne Again Shell): The default shell in most Linux distributions and widely used due to its simplicity and powerful features.
- Sh (Bourne Shell): The original Unix shell with basic functionalities.
- Ksh (Korn Shell): An advanced shell that combines features of Bourne and C shells.
- Csh (C Shell): Provides syntax similar to the C programming language.

Among these, Bash is the most popular and is extensively used for shell scripting.

Features of Shell Scripting

Shell scripting provides several important features that make it useful in system programming and automation:

- It allows automation of repetitive tasks, reducing manual effort.
- It supports variables, operators, and control structures similar to programming languages.
- It provides easy integration with system commands and utilities.
- It is simple to learn and implement, especially for beginners.
- It enables efficient handling of files, processes, and system operations.

Components of a Shell Script

A shell script consists of several basic components:

Shebang Line

The first line of a script specifies the interpreter to be used.

```
#!/bin/bash
```

This indicates that the script should be executed using the Bash shell.

Variables

Variables are used to store data values. They are declared without spaces around the assignment operator.

Example:

```
a=10
```

Input and Output

The read command is used to take input from the user, while echo is used to display output.

Operators

Shell scripting supports arithmetic, relational, and logical operators for performing calculations and decision-making.

Control Structures

Control structures determine the flow of execution:

- Conditional statements: if, if-else, case
- Looping statements: for, while, until

Working of Shell Script in Linux

In Linux, shell scripting is natively supported. The working of a shell script involves the following steps:

- First, a script file is created using a text editor such as nano or vi. The script begins with a shebang line followed by the required commands. After writing the script, it must be saved with a .sh extension.
- Next, execution permission must be provided to the file using the command:

```
chmod +x script.sh
```

This step is important because Linux enforces file permissions for security.
- Once permission is granted, the script is executed using:

```
./script.sh
```

- Internally, when the script is executed, the shell reads the shebang line to determine the interpreter. The Bash interpreter then processes the script line by line, executes each command, and displays the output on the terminal.

Working of Shell Script in Windows

Unlike Linux, Windows does not provide native support for Bash shell scripting. However, shell scripts can still be executed in Windows using external tools.

- One common method is using **Windows Subsystem for Linux (WSL)**. WSL allows users to run a Linux environment directly within Windows. After installing WSL and a Linux distribution such as Ubuntu, shell scripts can be executed in the same way as in Linux.
- Another method is using **Git Bash**, which provides a minimal Unix-like environment for running shell scripts. Additionally, tools like Cygwin can also be used to simulate a Linux environment.
- In Windows, the script execution process is similar, but it runs through the installed Bash environment instead of a native shell. Unlike Linux, execution permissions are generally not enforced strictly.

Shell scripting is an essential tool in Linux systems that enables users to automate tasks and manage system operations efficiently. With the availability of tools such as WSL and Git Bash, shell scripting can also be effectively utilized in Windows environments. Its simplicity and power make it an important skill for programmers and system administrators.

Experiment 1: Hello World Program

Aim

To display a simple message using shell scripting.

Algorithm

1. Start
2. Print a message
3. Stop

Program

```
#!/bin/bash  
echo "Hello, World!"
```

Output

Hello, World!

Experiment 2: Display Your Name**Aim**

To print a user-defined name.

Program

```
#!/bin/bash  
name="Abc"  
echo "My name is $name"
```

Experiment 3: Read and Display Input**Aim**

To take input from the user and display it.

Program

```
#!/bin/bash  
echo "Enter your name:"  
read name  
echo "Welcome $name"
```

Experiment 4: Addition of Two Numbers**Aim**

To perform addition of two numbers.

Program

```
#!/bin/bash  
echo "Enter two numbers:"  
read a b
```

```
sum=$((a + b))  
echo "Sum = $sum"
```

Experiment 5: Even or Odd Number

Aim

To check whether a number is even or odd.

Program

```
#!/bin/bash  
echo "Enter a number:"  
read n  
  
if [ $((n % 2)) -eq 0 ]  
then  
    echo "Even"  
else  
    echo "Odd"  
fi
```

Experiment 6: Positive or Negative Number

Aim

To check whether a number is positive or negative.

Program

```
#!/bin/bash  
echo "Enter a number:"  
read n  
  
if [ $n -ge 0 ]  
then  
    echo "Positive"  
else  
    echo "Negative"
```

fi

Experiment 7: Simple For Loop

Aim

To print numbers from 1 to 5.

Program

```
#!/bin/bash
```

```
for i in 1 2 3 4 5
```

```
do
```

```
    echo $i
```

```
done
```

Experiment 8: Simple While Loop

Aim

To print numbers from 1 to 5 using while loop.

Program

```
#!/bin/bash
```

```
i=1
```

```
while [ $i -le 5 ]
```

```
do
```

```
    echo $i
```

```
    i=$((i + 1))
```

```
done
```

Experiment 9: Simple Until Loop

Aim

To print numbers until a condition becomes true.

Program

```
#!/bin/bash
```

```
i=1

until [ $i -gt 5 ]
do
    echo $i
    i=$((i + 1))
done
```

Experiment 10: Basic Case Statement

Aim

To display message based on user choice.

Program

```
#!/bin/bash

echo "Enter a number (1-3):"
read n

case $n in
1) echo "One";;
2) echo "Two";;
3) echo "Three";;
*) echo "Invalid";;
esac
```

Practice Questions

1. Write a script to **find the square of a number**.
2. Write a script to **swap two numbers**.
3. Write a script to **print numbers from 1 to 10 using a loop**.
4. Write a script to **print your system date and time**.
5. Write a script to **find the largest of three numbers using if-elif**.
6. Write a script to **display the month name using case statement**.

7. Write a script to **print multiplication table of a number**.
8. Write a script to **find sum of first N natural numbers**.
9. Write a script to **reverse a number**.
10. Write a script to **check whether a number is palindrome**.
11. Write a script to **count number of digits in a number**.
12. Write a script to **print all even numbers up to N**.